



PROJECT NEXUS – PATTERN B

Enterprise LLM Token Optimization, Governance & Resilient Routing Engine

Anshul Vikas Kumaria

Summer Intern, EY AI Research

Executive Summary & Objectives

⚠ The Enterprise Challenge

- **Runaway Consumption:** Defaulting prompts to expensive flagship models.
- **Lack of Accountability:** Unallocated cloud spend without strict cost-center tracking.
- **Fragile Dependencies:** Upstream API dependencies prone to timeout bottlenecks.

🛡 The NEXUS Pattern B Solution

- **Modular Architecture:** Decouples prompt classification and WBS validation from transport.
- **Mandatory Perimeter Defense:** Rejects unbilled requests before gateway contact.
- **Automated Resilience:** Implements exponential backoffs, circuit breakers, and failovers.

Dynamic Multi-Tier Model Routing

src/orchestrator/router.py

Tier 1: gpt-4o-mini

Cost-optimized. Targets syntax checks, JSON parsing, and boilerplate extraction.

Tier 2: gpt-4o

Standard reasoning. Handles complex debugging and architecture design.

Tier 3: o1-preview

Frontier reasoning intelligence strictly for statutory audits and compliance.

```
class DynamicModelRouter:
    def classify_task(self, task_type: str, prompt: str):
        if any(re.search(p, prompt) for p in TIER_3_PATTERNS):
            return MODEL_TIERS["tier_3"] # o1-preview
        if any(re.search(p, prompt) for p in TIER_1_PATTERNS):
            return MODEL_TIERS["tier_1"] # gpt-4o-mini
        if any(re.search(p, prompt) for p in HIGH_COMPLEXITY_PATTERNS):
            return MODEL_TIERS["tier_2"] # gpt-4o
        return MODEL_TIERS["tier_2"] # default fallback
```

Governance & WBS Billing Enforcement

src/orchestrator/router.py

Header Standards & Validation

Accepts standardized corporate header aliases (X-EY-WBS-Element, X-WBS-Element, X-Cost-Center).

Validates strict matching format ensuring only billed traffic enters the pipeline.

Perimeter Defense Interception

Requests lacking a valid WBS identifier are intercepted immediately via Python exceptions.

Rejected at the API boundary with **HTTP 400** prior to touching upstream models.

```
def prepare_request(self, request: object) -> PreparedRequest:
    # Extract corporate billing headers
    wbs = next(
        (headers[name].strip() for name in (
            "X-EY-WBS-Element", "X-WBS-Element", "X-Cost-Center"
        ) if headers.get(name, "").strip()),
        ""
    )
    if not wbs:
        raise ValueError("Missing mandatory X-EY-WBS-Element")

    # Enforce strict WBS Regex contract
    if not re.fullmatch(r"WBS-[A-Z0-9_-]+", wbs):
        raise ValueError("Invalid X-EY-WBS-Element format")
```

Resilience, Retries & Local Fallback

src/orchestrator/gateway_client.py

Retry & Backoff

3 automatic retries with exponential backoff (1s, 2s, 4s) to absorb jitter.

Circuit Breaker

Trips after consecutive failures. Reroutes to local mock-gateway for zero downtime.

```
def send(self, request: GatewayRequest) -> GatewayResponse:
    if self.circuit_breaker.is_open():
        raise RuntimeError("Primary gateway circuit is open")

    try:
        payload = self._send_to(self.primary_url, body, headers)
        self.circuit_breaker.record_success()
        return GatewayResponse(payload["model"], "primary", payload, telemetry)

    except (HTTPError, URLError, TimeoutError, OSError):
        self.circuit_breaker.record_failure()
        return self._send_to_fallback(request_body, prepared.headers, telemetry)
```

Zero-Tolerance Telemetry Contract

src/schemas/telemetry.py

4

Strict Fields Enforced

timestamp, wbsElement, promptComplexity, tokenCount

22 / 22

Automated Tests Passed

pytest -v execution in 0.08s

```
from pydantic import BaseModel, ConfigDict, Field
from typing import Literal
from datetime import datetime

class TelemetryEvent(BaseModel):
    # Extra properties explicitly forbidden
    model_config = ConfigDict(extra="forbid", strict=True)

    timestamp: datetime
    # Billing field must match exact Regex
    wbsElement: str = Field(pattern=r"^WBS-[A-Z0-9_-]+$")
    promptComplexity: Literal["low", "high"]
    tokenCount: int = Field(ge=1)
```